- *N* is the total number of documents (figure 8.5 shows 25 documents in our index).

Similarly, the TF is calculated using this formula:

```
freq / (freq + k1 * (1 - b + b * dl / avgdl))
```

An explanation of how each variable is calculated is provided in the `details` section of the results. I recommend that you look at this section to check the application of the formulas by the engine to produce the score.

> **NOTE** What if there's no match? For example, what if we perform the search with "Lords" instead of "Lord"? We'd quickly find out that the results are empty. Try this in the code for yourself to view the results.

THE EXPLAIN API

Although we use the `explain` attribute to understand the mechanics of relevancy scoring, there's also an `_explain` API that provides insight into why a document matched (or not), in addition to providing scoring calculations. The query in the following listing uses an `_explain` endpoint with a document ID as the parameter to demonstrate this approach.

Listing 8.21 Explaining scores using the `_explain` endpoint

```
GET movies/_explain/14
{
  "query":{
    "match": {
      "title": "Lord"
    }
  }
}
```

This query is the same as that in listing 8.20, but this time, to explain the scores, we invoke the `_explain` endpoint instead of setting the `explain` flag on the `_search` endpoint.

Finally, let's misspell the word *Lord* as *Lords* in the `match` attribute in listing 8.21 and rerun the query. As you might expect, we don't get the same results. Instead, we get a clue:

```
{
  "_index" : "movies",
  "_type" : "_doc",
  "_id" : "14",
  "matched" : false,
  "explanation" : {
    "value" : 0.0,
    "description" : "no matching term",
    "details" : [ ]
  }
}
```